

CO1

ASSESSMENT TOOL-3

NAME : P. Ranjithkumar

DATE:27.07.2026

REG.NO: 192411119

Monday

COURSE CODE : CSA1103

COURSE NAME : OOAD

1. Smart Banking System – Object-Oriented Concepts

A **Smart Banking System** allows customers to open bank accounts, transfer funds, and pay utility bills. Object-Oriented Programming (OOP) helps represent the system using classes and objects.

(a) Object-Oriented Concepts

1. Class

A **class** is a blueprint or template used to create objects. It defines the properties and methods of an object.

Example:

`BankAccount` can be a class with properties such as account number, customer name, and balance.

2. Object

An **object** is a real instance of a class.

Example:

`Account101` is an object of the `BankAccount` class.

3. Message Passing

Message passing means communication between objects by calling their methods.

Example:

A `Customer` object sends a request to the `BankAccount` object to transfer money.

4. Encapsulation

Encapsulation means combining data and methods into one unit and restricting direct access to data.

Example:

The bank account balance can be private and changed only through methods like `deposit()` and `withdraw()`.

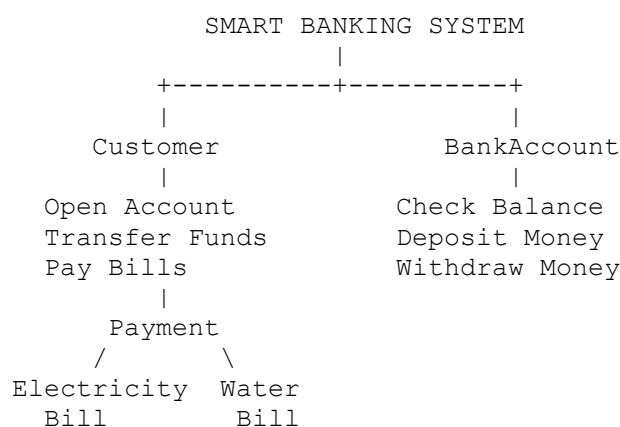
5. Polymorphism

Polymorphism means "many forms." The same method or operation can behave differently for different objects.

Example:

A `makePayment()` method can process different payment types such as electricity bill, water bill, or internet bill.

Simple Diagram



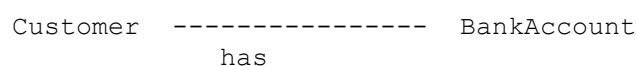
(b) Object Relationships

1. Association

Association represents a general relationship between two objects.

Example:

A customer has a bank account.

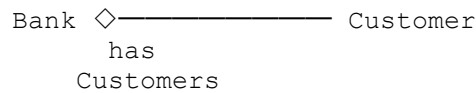


2. Aggregation

Aggregation represents a **weak whole-part relationship**. The parts can exist independently.

Example:

A bank has many customers. If the bank is closed, customer objects can still exist.

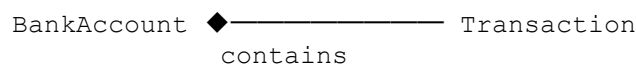


3. Composition

Composition represents a **strong whole-part relationship**. If the whole object is destroyed, its parts also cease to exist.

Example:

A bank account may contain transaction records that belong specifically to that account.



(c) Benefits of OOP Concepts

These concepts improve **reusability and maintainability** in a banking system.

- **Classes and objects** allow existing code to be reused.
- **Encapsulation** protects sensitive banking data.
- **Polymorphism** allows different payment methods to be handled easily.
- **Inheritance and relationships** reduce duplicate code.
- **Modularity** makes it easier to modify individual parts.
- Changes in one module have less effect on other modules.

Conclusion: OOP makes the Smart Banking System **secure, reusable, flexible, and easier to maintain**.

2. Smart Healthcare Management System – OOD Principles and SOLID

A **Smart Healthcare Management System** manages patients, doctors, appointments, medical records, and payments.

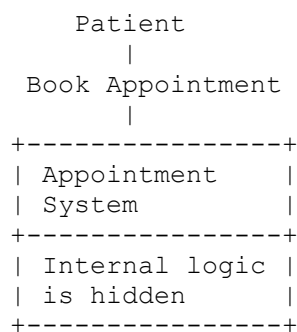
(a) Object-Oriented Design Principles

1. Abstraction

Abstraction means showing only the necessary information and hiding unnecessary implementation details.

Example:

A patient uses an `BookAppointment()` function without knowing how the appointment is stored internally.

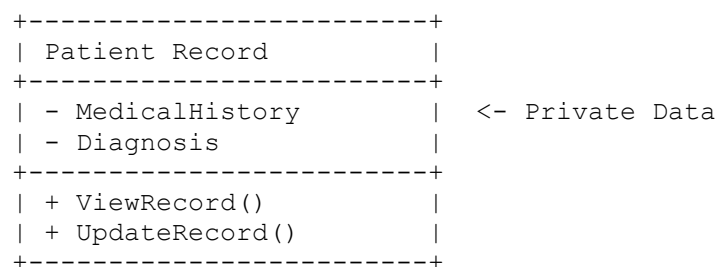


2. Encapsulation

Encapsulation combines data and methods into one class and protects the data from unauthorized access.

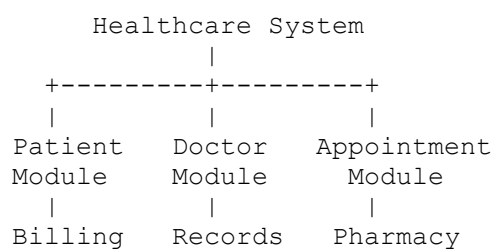
Example:

A patient's medical record should not be directly modified by everyone. Access should be provided through authorized methods.



3. Modularity

Modularity divides a large system into smaller independent modules.



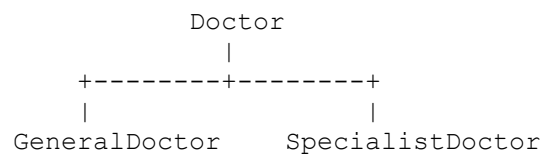
Each module performs a specific function.

(b) SOLID Principles

1. Liskov Substitution Principle (LSP)

LSP states that **objects of a child class should be replaceable for objects of its parent class without affecting the correctness of the program.**

Example:

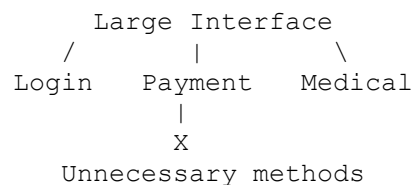


If the system expects a `Doctor`, both `GeneralDoctor` and `SpecialistDoctor` should work correctly wherever a `Doctor` is used.

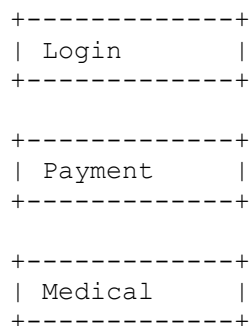
2. Interface Segregation Principle (ISP)

ISP states that a class should **not be forced to implement methods that it does not need.**

Instead of creating one large interface, create smaller and specific interfaces.



Better design:



Each class uses only the interface it requires.

(c) Benefits

These principles improve software quality and flexibility by:

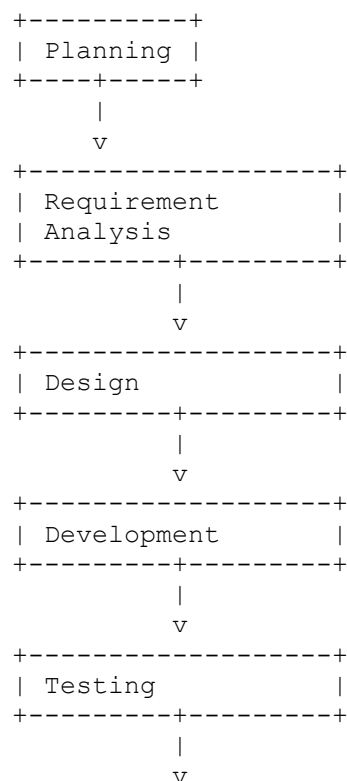
- Reducing unnecessary dependencies.
- Making code easier to understand.
- Protecting sensitive patient information.
- Allowing modules to be changed independently.
- Supporting future system expansion.
- Making testing and debugging easier.
- Improving code reuse.

Conclusion: These principles make healthcare software **flexible, maintainable, secure, and reliable**.

3. Online Food Delivery System – SDLC

The **Software Development Life Cycle (SDLC)** is a systematic process used to develop high-quality software.

(a) Phases of SDLC



```
+-----+
| Maintenance |
+-----+
```

1. Planning

The project goals, scope, resources, time, and budget are identified.

2. Requirement Analysis

The development team collects and analyzes user requirements.

Example:

Customers should be able to search restaurants, order food, make payments, and track deliveries.

3. Design

The system architecture, database, user interface, and software components are designed.

4. Development

Programmers write the actual source code based on the design.

5. Testing

The software is tested to identify and fix errors and defects.

6. Maintenance

After deployment, the software is updated, improved, and corrected according to user needs.

(b) Purpose of Each Phase

Phase	Purpose
Planning	Defines project goals and resources
Requirement Analysis	Understands user needs
Design	Creates system structure
Development	Converts design into code
Testing	Finds and fixes errors
Maintenance	Supports and improves the software

(c) How SDLC Delivers Reliable Software

SDLC provides a **step-by-step and organized development process**. Requirements are analyzed before development, the system is properly designed, and testing identifies errors before delivery. Maintenance ensures the software continues to work after deployment.

Conclusion: SDLC helps deliver software that is **reliable, high-quality, cost-effective, and easy to maintain**.

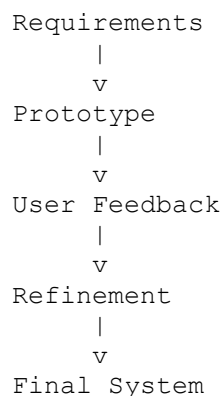
4. Smart Agriculture Monitoring System – SDLC Models

A Smart Agriculture Monitoring System may have changing requirements because farmers may request new features such as weather monitoring, soil monitoring, and crop prediction.

(a) SDLC Models

1. Prototype Model

In the Prototype Model, a quick working model is developed first. Users provide feedback, and the system is improved.

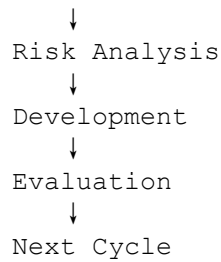


Advantage: Useful when requirements are unclear.

2. Spiral Model

The Spiral Model combines iterative development with risk analysis. The system is developed through repeated cycles.

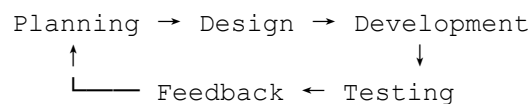
Planning



Advantage: Suitable for large and high-risk projects.

3. Agile Model

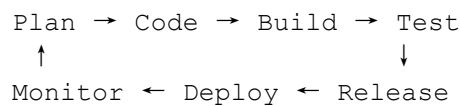
Agile develops software in small iterations called **sprints**. Customer feedback is regularly collected.



Advantage: Handles frequently changing requirements effectively.

4. DevOps Model

DevOps combines **Development and Operations**. It focuses on continuous development, testing, integration, deployment, and monitoring.



Advantage: Provides faster and continuous software delivery.

(b) Prototype Model vs Agile Model

Prototype Model	Agile Model
Focuses on building an early prototype	Focuses on developing working software incrementally
Mainly useful for unclear requirements	Useful for frequently changing requirements
Customer feedback improves the prototype	Customer feedback is collected in every iteration
Prototype may be discarded	Each sprint produces useful software

Prototype Model	Agile Model
Suitable for understanding user needs	Suitable for continuous development

(c) Recommended Model

For the **Smart Agriculture Monitoring System**, the **Agile Model** is the most suitable.

Reasons:

- Customer requirements change regularly.
- New agricultural features can be added easily.
- Farmers can provide continuous feedback.
- The system can be developed in small iterations.
- Problems can be identified early.
- New technologies and sensors can be integrated gradually.

Conclusion: Agile is recommended because it provides **flexibility, continuous feedback, faster development, and easy adaptation to changing requirements**.

5. Smart Transport Management System – UML

(a) Importance of UML

UML (Unified Modeling Language) is a standard visual language used to model and design software systems.

UML is important because it:

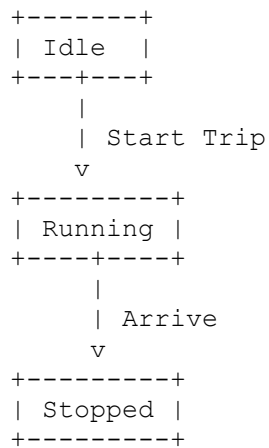
- Provides a visual representation of the system.
 - Helps developers understand system structure and behavior.
 - Improves communication between developers and customers.
 - Helps identify design problems early.
 - Provides documentation for future maintenance.
 - Supports object-oriented software development.
-

(b) UML Diagrams

1. State Diagram

A State Diagram shows the different states of an object and the transitions between states.

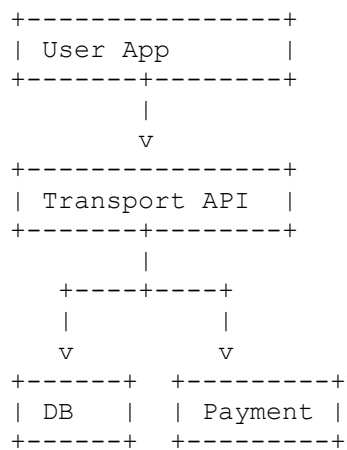
Example: Transport Vehicle



It shows how a vehicle changes its state during transportation.

2. Component Diagram

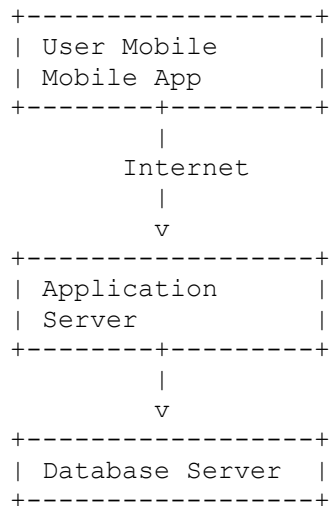
A Component Diagram shows the major software components and their relationships.



Example: Components may include User App, Transport API, Database, and Payment Service.

3. Deployment Diagram

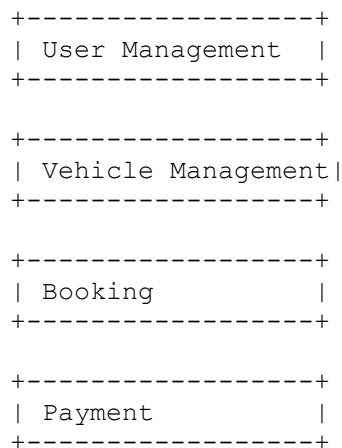
A Deployment Diagram shows the physical arrangement of hardware and software components.



It shows where the software is deployed physically.

4. Package Diagram

A Package Diagram organizes related classes and components into packages.



It helps organize a large system into manageable groups.

(c) UML and Stakeholder Communication

UML models improve communication because diagrams are easy to understand compared to long technical descriptions. Customers can understand system functionality, while developers can understand system structure and behavior.

For example:

- Customers understand **what the system does**.

- Developers understand **how the system works**.
- Testers understand **what needs to be tested**.
- Managers understand **the overall system structure**.

Conclusion: UML provides a **common visual language** that improves communication and reduces misunderstandings among stakeholders.

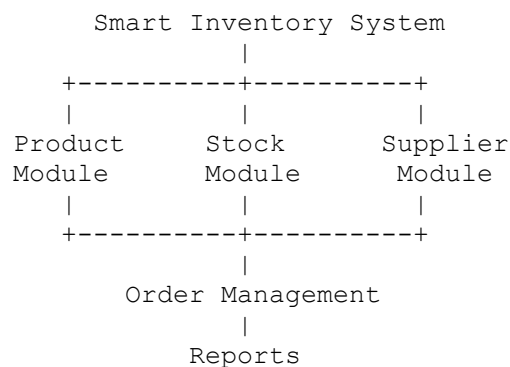
6. Smart Inventory Management System – Modular Design

A Smart Inventory Management System manages products, stock, suppliers, orders, and reports.

(a) Modular Software Design

Modular software design is the process of dividing a large software system into smaller, independent modules. Each module performs a specific task.

Example



For example, the **Stock Module** manages stock quantity, while the **Supplier Module** manages supplier information.

(b) Procedural vs Object-Oriented Programming

Procedural Programming	Object-Oriented Programming
Program is divided into functions	Program is divided into objects/classes
Focuses on procedures and functions	Focuses on objects and data
Data may be shared between functions	Data can be protected using

Procedural Programming

Less suitable for large complex systems

Reusability is comparatively lower

Example: C

Object-Oriented Programming

encapsulation

Suitable for large complex systems

Provides better code reuse

Examples: Java, C++, Python

Simple Diagram

Procedural Approach

```
Data
|
+--> Function 1
|
+--> Function 2
|
+--> Function 3
```

Object-Oriented Approach

```
+-----+
| Inventory Object |
+-----+
| Product Data    |
| Stock Data      |
+-----+
| Add Product()   |
| Update Stock()  |
| Check Stock()   |
+-----+
```

(c) Benefits of Modularity

1. Scalability

New modules can be added without changing the entire system.

Example:

A new AI Stock Prediction Module can be added to the existing inventory system.

2. Maintainability

Each module can be modified or repaired independently.

Example:

Changes in the payment module do not affect the product module.

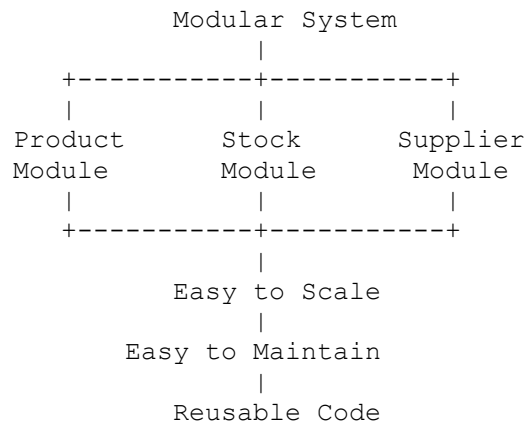
3. Reusability

Existing modules can be reused in other applications.

Example:

The Supplier Management Module can be reused in another business management system.

Overall Diagram



Conclusion: Modular design divides a complex system into smaller manageable units. It improves **scalability, maintainability, reusability, testing, and software quality**.